

Requirements Engineering & Planning

Overview

6 Sept 2026

- translate goals into machine learning engineering requirements
- explain the main characteristics of training pipeline, inference pipeline, and patterns for serving models
- examine approaches to design ML systems to handle the inevitable mistakes and failures that occur in real-world AI applications

Readings

How We Built Infrastructure to Run User Forecasts at Spotify

Molly Zhu, Insights Manager. (n.d.). *How we built infrastructure to run user forecasts at Spotify* | *Spotify Engineering*. Spotify Engineering.

<https://engineering.atspotify.com/2022/6/how-we-built-infrastructure-to-run-user-forecasts-at-spotify>

Common pitfalls when building generative AI applications

Huyen, C. (2025, January 16). Common pitfalls when building generative AI applications.

Chip Huyen. <https://huyenchip.com/2025/01/16/ai-engineering-pitfalls.html>

Tools Needed This Week:

Apache Airflow

<https://airflow.apache.org/docs/apache-airflow/stable/index.html>

PyTest

<https://docs.pytest.org/en/stable/>

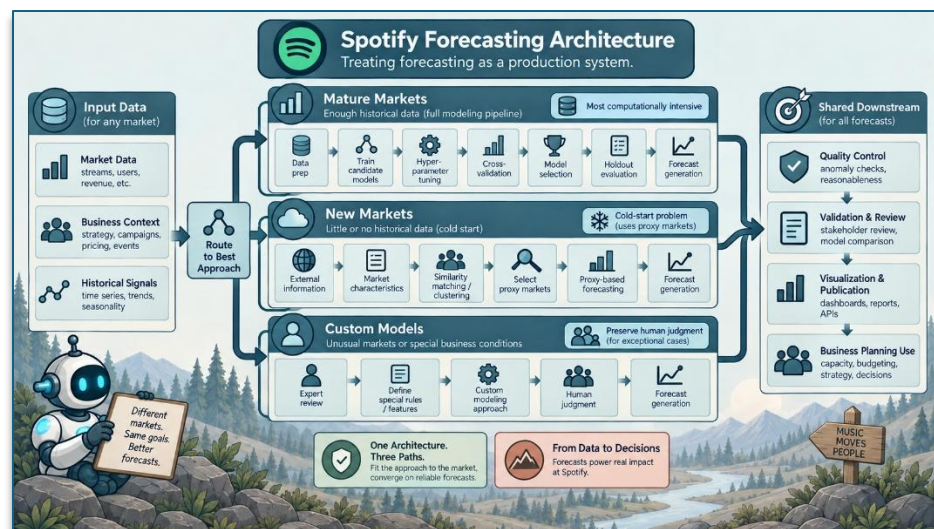
How We Built Infrastructure to Run User Forecasts at Spotify	
Overview	Scaling User Forecasting Infrastructure At Spotify. Spotify redesigned its user forecasting infrastructure to support automated, high-quality forecasts that could run weekly or on demand across a rapidly expanding global business.
Purpose And Context	Spotify needed a forecasting system that could support rapid international growth. • the company operated across more than 180 countries • user forecasts were important for: ○ business planning ○ measuring company health • older forecasting processes were too slow and difficult to scale • the goal was to generate reliable forecasts for individual markets • forecasts needed to run automatically each week • analysts also needed the ability to generate forecasts on demand
Core Design Idea	Forecasting was treated as a production system rather than only a statistical modeling problem. • the system combined: ○ automated modeling ○ large-scale computation ○ data quality checks ○ model validation ○ visualization ○ business knowledge ○ human review

Work that could previously take months on one machine could be completed within hours.

Forecasting Architecture

Spotify divided markets into three main forecasting categories:

- mature markets
- new markets
- custom models



ChatGPT 5.6 Sol

- different market conditions required different modeling strategies
- the amount of historical data determined which approach was most appropriate

Mature Markets.

- mature markets had enough historical data for traditional time-series forecasting
- models could learn:
 - long-term trends
 - seasonal patterns
 - holiday effects

- changes in growth
 - data was cleaned before training
 - unusual one-time events were removed when they were unlikely to repeat
- this prevented exceptional historical events from being treated as normal future patterns

Mature Market Pipeline.

- the mature-market workflow included:
 - data preparation
 - model training
 - hyperparameter tuning
 - cross-validation
 - model selection
 - holdout evaluation
 - forecast generation
 - quality control
 - publication
- this was the most computationally expensive part of the system

Model Selection.

- Spotify evaluated many combinations of:
 - markets
 - models
 - hyperparameters
 - validation periods

- candidate models were tested in parallel
 - each configuration produced an error measurement
 - the strongest models were selected using weighted error
 - final candidates were evaluated against a holdout dataset
 - the holdout data was not used during model tuning
- this reduced the risk of choosing models that only performed well on data already used during development

New Markets.

- new markets had little or no historical user data
- this created a cold-start forecasting problem
- traditional time-series models could not be used effectively
- Spotify instead used proxy markets

Proxy-Based Forecasting.

- external information was used to identify existing markets that resembled a new market
- data included:
 - macroeconomic information
 - music-related information
 - market characteristics
- clustering models identified similar markets
 - these existing markets became proxies for the new market
- Spotify then used historical behavior from the proxy markets to estimate likely growth in the new market

Cold-Start Lesson.

- when historical data is insufficient:
 - the problem may need to be reformulated

	• Spotify did not force a traditional time-series model onto limited data • instead, the system transferred information from similar markets with known histories Custom Models. • some markets did not fit the mature-market or new-market workflows • these markets required custom handling • custom cases could include: ○ experimental models ○ direct data scientist review ○ special business information ○ major upcoming campaigns ○ unusual market conditions • this preserved human judgment when automated models did not contain enough context
Infrastructure And Scaling	Spotify used different infrastructure for different computational workloads. • Google Cloud Dataflow and Apache Beam: ○ used for large parallel workloads • containerized Python jobs on Kubernetes were used for smaller tasks • the system did not parallelize every operation

When Parallelization Helps	Distributed computing reduces runtime for large workloads. - distributed systems: - introduce overhead and complexity - parallelization: - most useful when the workload is large enough to justify the added infrastructure - Spotify found that better modeling decisions could sometimes reduce computation more effectively than adding more hardware
Reducing Computational Cost	Computational cost could be reduced by: - narrowing the hyperparameter search space - choosing models appropriate for the problem - avoiding unnecessary combinations - using parallel infrastructure only when beneficial - modeling efficiency and infrastructure efficiency were treated as related problems
Quality Control	Forecast generation was followed by automated quality checks. - results were distributed through internal systems - new forecasts could be compared with previous official forecasts - visualizations helped identify unusual changes or trends - visual inspection remained important because some problems are easier to recognize visually than through numerical metrics alone

Human Intervention	Historical data cannot represent every future event. • business stakeholders may know about future conditions that the model cannot observe • examples include: ○ marketing campaigns ○ sales campaigns ○ product changes ○ market-specific events • the system allowed this information to influence the final forecast • automation therefore supported human judgment rather than completely replacing it
Engineering Lessons	Core Logic Requires Strong Controls. • critical forecasting logic received: ○ testing ○ peer review ○ documentation ○ release controls ○ integration testing • less critical components could remain more flexible ○ examples included: ▪ visualization ▪ email-generation code • this allowed faster iteration without weakening the reliability of the core forecasting system

Scaling Is Not Always The Best Solution	More computing resources do not automatically produce a better system. - distributed infrastructure adds complexity - a smaller hyperparameter space may reduce runtime more effectively - a better model choice may reduce computation more effectively - engineering decisions should balance: - speed - cost - complexity Simplicity Matters. - small improvements in predictive accuracy may not justify major increases in system complexity - new functionality does not always need to become part of the core infrastructure - some features can remain separate extensions - production systems should prioritize: - reliability - maintainability - clarity - scalability - reasonable predictive performance
Common Pitfalls When Building Generative AI Applications	
Overview	Chip Huyen identifies six recurring mistakes that appear when teams build generative AI products. The central argument is that

	building a successful AI application requires much more than choosing a capable model. Many failures come from product design, engineering decisions, evaluation methods, and organizational strategy. Generative AI should be treated as one possible tool for solving a problem. It should not become the problem-solving strategy itself. The easiest stage is often building an impressive demonstration. The difficult stage is creating a reliable product that users actually need.
Core Idea	Building a convincing AI demo is relatively easy. • building a reliable and useful AI product is much harder • many failures come from: ○ product design ○ evaluation, ○ engineering complexity ○ poor use-case selection The central principle is to start with the problem, not with generative AI.
Pitfall 1: Using Generative AI When It Is Not Needed	Problem. • teams sometimes choose generative AI before deciding whether it is the best solution • a task may already have a simpler statistical, optimization, or rule-based solution • an LLM can solve a problem without being the most efficient way to solve it

	Example. • an energy scheduling system used an LLM to decide when household activities should occur • a conventional scheduling or optimization method might have produced similar results with less complexity Lesson. • first determine what the problem requires • then decide whether generative AI provides a meaningful advantage
Pitfall 2: Blaming The Model For A Bad Product	Problem. • a technically capable model can still produce an unsuccessful product • model quality is only one part of the user experience • users may want something different from what developers initially expect Examples. • meeting-summary users cared more about identifying action items than about receiving the perfect summary • job-search users wanted guidance on how to improve their fit, not only a prediction of whether they matched a role • chatbot users often responded better when the interface suggested useful questions instead of presenting an empty text box Lesson.

	• product design should be based on real user behavior • model accuracy does not automatically create user value
Pitfall 3: Starting With Too Much Complexity	Problem. • AI teams often introduce advanced infrastructure before proving that it is necessary • common examples include: ○ agent frameworks ○ vector databases ○ fine-tuning ○ complex caching systems ○ large orchestration frameworks Why This Matters. • additional abstractions make systems harder to understand and debug • frameworks can also hide prompts, internal logic, and unexpected behavior • when something fails, the source of the problem becomes harder to identify Lesson. • start with the simplest architecture that can test the idea • add complexity only when a specific limitation justifies it
Pitfall 4: Overestimating Early Success	Problem. • AI prototypes can reach apparently strong performance very quickly

	• the final improvements needed for production are much harder • Huyen describes teams reaching roughly 80 percent of the desired experience quickly, then spending much longer pushing toward production quality The Difficult Final Stage. • remaining problems often include: ○ hallucinations ○ edge cases ○ latency ○ incorrect tool use ○ misunderstood user intent ○ inconsistent outputs ○ security and privacy ○ provider failures ○ model changes ○ safety and compliance Lesson. • prototype success does not mean production readiness • the last portion of system quality may require more engineering effort than the initial prototype
Pitfall 5: Replacing Human Evaluation Too Early	Problem. • automated evaluation is useful, but it cannot fully replace human judgment • LLM-as-a-judge systems can evaluate large numbers of responses • however, the evaluator is also a model

	• its decisions depend on prompts, criteria, and model behavior Human Review. • strong teams still inspect real outputs manually • human evaluation can reveal: ○ unexpected failure patterns ○ new user behavior ○ changes that metrics do not capture ○ cases where automated evaluation disagrees with human judgment Lesson. • automated evaluation should scale human evaluation rather than eliminate it • evaluation should continue throughout the life of the system
Pitfall 6: Choosing Use Cases Without A Strategy	Problem. • organizations can build many AI applications without creating meaningful business value • employees often identify problems from their own daily workflows • those problems may be real, but they are not necessarily the highest-value opportunities for the organization Common Result. • companies may create many: ○ chatbots ○ text-to-SQL tools ○ coding assistants

	○ internal productivity tools • individual projects may work while the overall AI investment produces little measurable return Lesson. • use cases should be evaluated according to: ○ business impact ○ cost ○ technical feasibility ○ expected return ○ strategic importance AI adoption needs an organizational strategy, not simply a collection of experiments.
The Demo-To-Product Gap	Foundation models make prototypes unusually easy to build. • a small amount of code and prompting can produce an impressive demonstration • production systems face a much larger set of requirements • they must remain reliable across different: ○ users ○ inputs, ○ environments ○ failure conditions • they must also handle: ○ security

	○ privacy ○ latency ○ safety ○ compliance ○ model changes ○ unexpected behavior This creates a large gap between showing that an AI system can work and proving that it can work reliably.
Recommended Engineering Approach	Start With The Problem. • understand the user need before selecting the technology Use The Simplest Viable System. • avoid adding infrastructure until there is evidence that it is required Evaluate With Real Users. • user behavior often reveals needs that initial requirements miss Keep Humans In The Evaluation Loop. • automated metrics should be checked against human judgment Expect The Final Improvements To Be Expensive. • production reliability requires substantial work beyond the prototype Choose Use Cases Strategically.

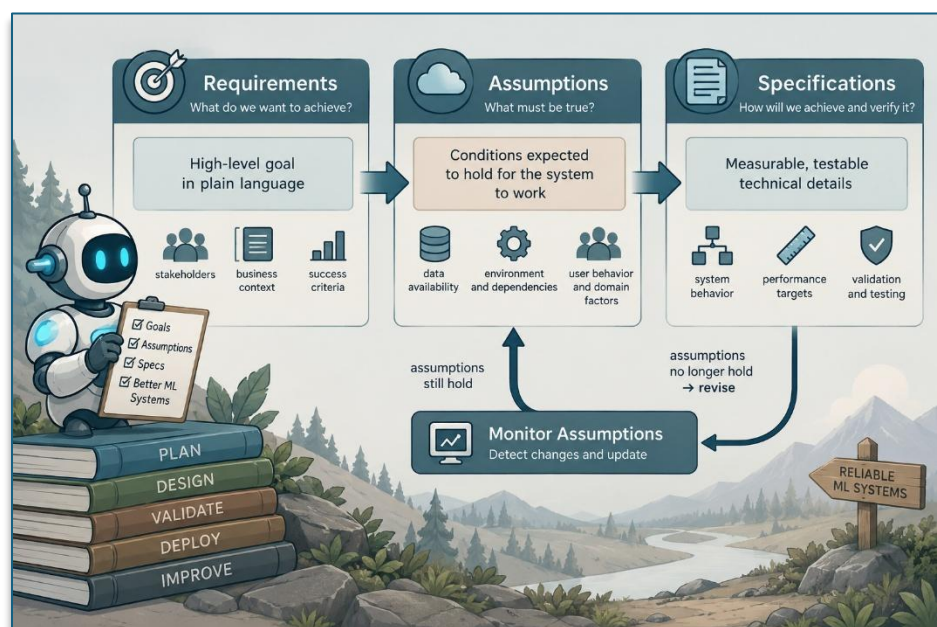
	• AI projects should be selected for their potential value rather than for technological novelty
Key Takeaway	The main challenge in AI engineering is not getting a model to produce an impressive response. • the real challenge is building a system that is: ○ useful ○ reliable ○ understandable ○ worth maintaining • successful teams begin with the problem ○ they keep the architecture simple ○ they evaluate with humans and real users ○ they expect production quality to require significant additional work ○ they choose AI projects according to value rather than novelty Generative AI is therefore best treated as one engineering tool within a broader product and systems strategy.
Async	
Requirements Engineering and Planning for Mistakes	
Overview	This unit moves from model development toward production engineering. • the main goal:

- translate business and system goals into clear engineering requirements
- the unit also focuses on designing systems that can tolerate:
 - mistakes
 - failures
 - unexpected conditions
- the central idea:
 - production ML systems should assume that failures will occur

The engineering task is to detect those failures early, limit their impact, and recover safely.

Requirements, Assumptions, And Specifications

ML systems can be designed using the classic requirements, assumptions, and specifications framework.



ChatGPT 5.6 Sol

Requirements.

- define what the system must achieve
- examples include:
 - acceptable prediction quality
 - maximum latency
 - reliability targets
 - business constraints
 - safety requirements

Assumptions.

- describe conditions that the system expects to remain true
- examples include:
 - input data follows expected distributions
 - features are available
 - upstream services remain operational
 - users behave within expected patterns

Specifications.

- translate requirements and assumptions into technical behavior
- examples include:
 - data validation rules
 - deployment procedures
 - monitoring thresholds
 - rollback conditions
 - model performance limits

Why This Matters	Many production failures occur when assumptions are treated as permanent facts, • an ML system should make important assumptions explicit • those assumptions can then be monitored and tested
Training And Inference Pipelines	Production ML systems usually contain separate training and inference pipelines. Training Pipeline. • responsible for creating and updating models • common stages include: ○ data collection ○ data validation ○ feature preparation ○ model training ○ evaluation ○ model registration Inference Pipeline. • responsible for serving predictions to users or downstream systems • common stages include: ○ receiving input ○ validating input ○ creating features ○ running the model ○ returning predictions ○ logging results

Key Difference	The training pipeline determines how models are created. - the inference pipeline determines how models behave in production - a model can perform well during training while still fail during inference because of: - data changes - serving problems - infrastructure failures
Safe Model Deployment	New model versions should not immediately replace the current production system without testing. deployment patterns allow teams to reduce the risk of introducing failures
Blue-Green Deployment	Two production environments are maintained. - one environment serves the current model - the second environment contains the new model version - traffic can be switched between environments - if the new model fails, traffic can quickly return to the previous environment Main Benefit. - fast rollback with minimal disruption
Canary Release	A small percentage of production traffic is routed to the new model. - most users continue using the existing version - engineers monitor the new model before increasing traffic

	Main Benefit. • limits the number of users affected by an unexpected failure
Shadow Testing	The new model processes real production inputs in parallel with the existing model. • users continue receiving predictions from the existing system • the new model's outputs are recorded but not shown to users Main Benefit. • allows testing with realistic traffic without affecting production behavior
Experimentation Strategies	Production changes should be evaluated before full deployment. • two important approaches are online experiments and simulated or offline experiments
A/B Testing	Users are divided between different system versions. • one group may receive predictions from the current model • another group receives predictions from the new model • outcomes can then be compared using selected metrics Possible Metrics. • prediction quality • user engagement • conversion • latency • business outcomes

	- error rates Main Purpose. - measure whether a model change produces meaningful improvement in the real system
Offline And Simulated Testing	Online experimentation is not always appropriate. - it may be: - too risky - too expensive - too slow - ethically inappropriate - operationally difficult - offline experiments can evaluate new behavior using historical or simulated data before exposing users to the change Main Purpose. - reduce risk before production deployment
Planning For Mistakes	Reliable ML systems are designed around the expectation that problems will occur. - the objective is not to eliminate every possible failure - the objective is to anticipate likely failures and respond safely
Common Failure Modes	Bad Data. - inputs may be: - missing - malformed,

- corrupted
- outside expected ranges

- data validation can detect many of these problems before predictions are produced

Data Drift.

- the statistical distribution of input data may change over time
- the model may begin receiving data that differs from its training data

Concept Drift.

- the relationship between inputs and outcomes may change
- even valid input data may produce less accurate predictions because the real-world process has changed

Model Errors.

- models can produce incorrect or unexpected outputs
- performance should therefore be monitored after deployment

Adversarial Inputs.

- users or external systems may intentionally provide unusual inputs designed to manipulate model behavior
- systems should include:
 - validation
 - monitoring
 - security controls

	Infrastructure Failures. • models depend on: ○ databases ○ APIs ○ networks ○ compute resources ○ other services An outage in any of these components can interrupt inference.
Failure Mitigation	Production systems should include mechanisms for both detection and recovery. Detection. • monitor data quality • track model performance • watch latency and error rates • detect changes in input distributions • log abnormal behavior Mitigation. • roll back problematic models • route traffic to a stable version • reject invalid inputs • use fallback behavior • retrain models when conditions change • isolate failures before they spread

Engineering Principle	A production ML system should be designed for failure rather than assuming ideal conditions. • new models should be introduced gradually • changes should be tested before full deployment • critical assumptions should be monitored • failures should have defined recovery procedures
Key Takeaway	Requirements engineering connects ML goals to concrete production behavior. • requirements: ○ define what the system must accomplish • assumptions: ○ define the conditions the system expects • specifications: ○ define how those expectations are implemented and tested • training and inference pipelines: ○ should be treated as separate engineering concerns • blue-green deployments, canary releases, and shadow testing: ○ reduce deployment risk • A/B testing and offline experiments: ○ help determine whether changes are beneficial before full adoption • production systems should also anticipate: ○ bad data ○ drift ○ model errors ○ adversarial inputs

	○ infrastructure outages The broader lesson is that reliable ML engineering is not based on preventing every mistake. • it is based on detecting problems quickly, limiting their impact, and recovering safely
REQ-ASM-SPEC Model	
Core Idea	Requirements engineering converts ML goals into production requirements. Reliable systems are designed with the expectation that failures will occur.
Requirements Framework	Requirements. • define what the system must accomplish Assumptions. • define the conditions expected to remain true Specifications. • translate requirements and assumptions into technical rules that can be implemented and tested
Production Pipelines	Training Pipeline. • creates and updates the model Inference Pipeline. • serves predictions in production

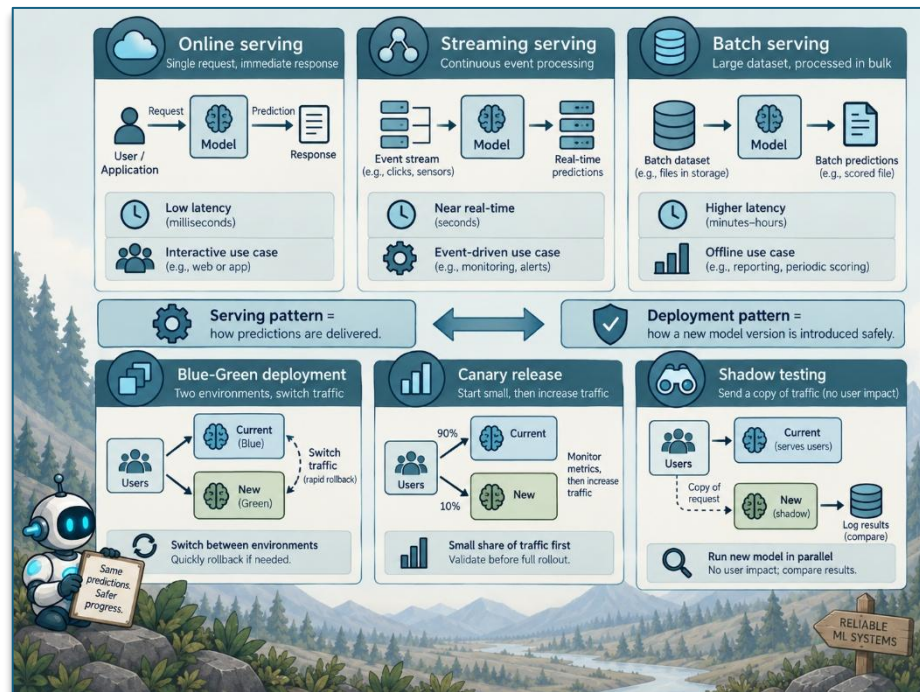
	The two pipelines have different risks and should be engineered separately.
Safe Deployment	Blue-Green Deployment. • maintains two production environments for rapid rollback Canary Release. • routes a small amount of traffic to a new model before wider deployment Shadow Testing. • runs a new model on live traffic without exposing its predictions to users
Safe Experimentation	A/B Testing. • compares model versions using real users and production metrics Offline Or Simulated Testing. • tests changes without exposing users when online experimentation is too risky or expensive
Planning For Failure	Production systems should anticipate: • bad or malformed data • data and concept drift • model errors • adversarial inputs • infrastructure outages

Detection And Recovery	Detection. • monitor data • model performance • latency • errors • changing distributions Recovery. • roll back models • use stable fallbacks • reject invalid inputs • retrain when conditions change • isolate failures before they spread
Key Takeaway	Reliable ML engineering does not depend on preventing every failure. • it depends on anticipating: ○ failure ○ detecting problems quickly ○ limiting their impact ○ recovering safely
Training and Inference Pipelines	
Core Difference	Training and inference pipelines serve different purposes and have different engineering requirements. • training builds the model • inference uses the model to make predictions • keeping both pipelines consistent is a major production challenge

Training Pipeline	Purpose. • process large amounts of historical data and build models offline Typical Workloads. • reading terabytes of data • distributed training across clusters • large batch computations • model evaluation Primary Priorities. • throughput • cost efficiency • model quality • completion within the required training schedule Latency. • training usually does not require millisecond response times • a job taking three hours instead of two may be acceptable if it still completes within the required cadence
Inference Pipeline	Purpose. • produce predictions when new data arrives Typical Workloads. • fetch input data • preprocess features • run model inference • postprocess predictions

	• return results Primary Priorities. • low latency • high reliability • consistent performance • scalability under load Latency. • inference may be part of the application's critical path • users may be waiting directly for the result • response times are often measured in milliseconds
Training Versus Inference	Training. • offline • large-scale computation • optimized for throughput and cost Inference. • online or production-facing • serves new requests • optimized for latency and reliability
Feature Parity	Training and inference must process features in the same way. • if training calculates a user's average spending over previous months, inference must reproduce that calculation using information available at prediction time

Train-Serve Skew	Train-serve skew occurs when features are computed differently during training and inference. - the model then receives production inputs that differ from what it learned during training - this effectively causes the model to solve a different problem at serving time
Reducing Train-Serve Skew	Shared Transformation Code. - use the same feature-processing logic in both pipelines Feature Stores. - store and serve consistent feature definitions for training and inference
Data Leakage	Training data must not include information that would be unavailable when a real prediction is made. - future information can artificially improve training results - the deployed model will not have access to that information - preventing leakage must therefore be part of training pipeline design
Model Serving Patterns	Online Serving. - provides predictions through a real-time API - best suited for: - interactive applications - immediate decisions - user-facing predictions



ChatGPT 5.6 Sol

Main Requirements.

- low latency
- high availability
- reliable scaling

Streaming Serving.

- processes a continuous flow of incoming data
- common examples include:
 - sensor feeds
 - click streams
 - IoT telemetry
 - application logs

Main Requirements.

- continuous processing
- integration with streaming infrastructure

	• rapid response to incoming events Batch Serving. • processes many predictions asynchronously • common examples include: ○ daily user scoring ○ periodic reports ○ scheduled analytics Main Benefits. • lower cost • efficient processing of large workloads • no strict real-time latency requirement
Choosing A Serving Pattern	Online. • use when an immediate response is required Batch. • use when predictions can be produced periodically • often simpler and less expensive Streaming. • use when data arrives continuously and must be processed as events occur Hybrid Systems. • some systems combine multiple serving patterns • for example:

	○ an online model provides an immediate prediction ○ a nightly batch process performs deeper analysis The appropriate architecture depends on application requirements, cost, and latency needs.
Online Inference Requirements	Service Level Objectives. • latency requirements should be defined as a distribution rather than only as a single average value • the system must account for both typical and slower requests Operational Planning. • online systems should plan for: ○ timeouts ○ traffic spikes ○ back pressure ○ slow dependencies ○ overloaded services Fallbacks. • systems should define what happens when latency or reliability requirements cannot be met • possible responses include: ○ returning a cached result ○ using a simpler model ○ using a default prediction ○ failing safely instead of waiting indefinitely

Key Takeaway**Training and inference pipelines optimize for different goals.**

- **training** emphasizes:
 - large-scale data processing
 - throughput
 - cost efficiency
- **inference** emphasizes:
 - latency
 - reliability
 - scalability
- **feature processing:**
 - must remain consistent between both pipelines
- **shared transformations and feature stores:**
 - can reduce train-serve skew
- **training:**
 - must also prevent data leakage from future information
- **online, streaming, and batch serving:**
 - each support different application needs
- **reliable production systems** choose the serving pattern that matches the required:
 - latency
 - workload
 - cost

Right-Sizing, Batching Requests, Autoscaling	
Core Idea	Production ML systems must balance performance, cost, and deployment risk. • inference should use hardware efficiently • new models should be introduced gradually • experiments should have predefined metrics and decision rules • important ML components should also have written requirements
Managing Inference Cost	Cost Per Prediction. • cloud inference consumes: ○ CPU ○ GPU ○ memory ○ network resources • cost should therefore be tracked as a production metric • a useful measure is: ○ cost per 1,000 predictions ○ a new model may cost more if it provides enough additional value • the important point is that the tradeoff should be visible and intentional Right-Sizing Hardware. • not every model requires a GPU • smaller workloads may run more cheaply on CPUs • large transformer models may require one or more GPUs • hardware should match:

	○ model size ○ traffic volume ○ latency requirements ○ cost constraints Batching Requests. • multiple inference requests can sometimes be processed together • deep learning models are especially efficient when processing batches through vectorized operations Tradeoff. • batching increases throughput and lowers cost • but requests may wait briefly while a batch is formed • this can slightly increase latency Autoscaling. • traffic changes over time • cloud systems can automatically: ○ add instances when demand increases ○ remove instances when demand falls • autoscaling improves resource efficiency ○ but it can also create cold-start problems
Latency Optimization	Warm Pools. • new instances may take time to start and load a model • this creates a cold start Common Solutions.

	• keep a minimum number of containers running • preload models before traffic arrives • periodically keep serverless services active Caching. • repeated computations do not always need to be performed again Feature Cache. • stores expensive feature-engineering results • useful when user or entity features change slowly Prediction Cache. • stores recent model outputs • useful when identical or similar requests occur repeatedly
Model Optimization	Models can sometimes be made faster without significant accuracy loss. Quantization. • reduces numerical precision • can reduce model size and speed up inference Distillation. • trains a smaller model to imitate a larger model The smaller model is then deployed for cheaper and faster inference.

Autonomous Driving	
Core Idea	Requirements engineering makes ML system behavior explicit before deployment. - the framework separates: - requirements - specifications - assumptions The autonomous driving example shows how each part defines a different aspect of system behavior.
Autonomous Driving Example	Requirement. - prevent the vehicle from leaving its lane unless the movement is intentional - intentional movement may include: - turn signal use - driver input Specification. - define exactly how the system should achieve the requirement - possible specifications include: - detect lane markings using computer vision - measure deviation from the lane center - apply corrective steering - respond within a defined amount of time - operate under defined conditions such as highway driving above 60 miles per hour

	Assumptions. • define what must remain true for the system to work • examples include: ○ camera is functioning ○ lane markings are visible ○ road conditions allow lane detection • if heavy snow covers the lane markings, an important assumption fails • the system may then be unable to satisfy the original requirement
Applying The Framework To ML	The same structure can be applied to ordinary machine learning systems. Churn Prediction Example. Requirement. • proactively retain customers who are likely to churn Specification. • predict churn with a defined performance threshold • trigger an intervention after a high-risk prediction • possible interventions include: ○ email ○ special offer ○ sales follow-up • the intervention should occur within a defined period after prediction Assumptions.

	• customers respond to offers similarly to past customers • customer activity data is updated reliably • sales teams follow up when required • these assumptions include both technical and human components Why Assumptions Matter. • a system can fail even when the model itself works correctly • the model may depend on: ○ data availability ○ user behavior ○ business processes ○ human follow-up ○ external systems • changes in any of these can break the connection between prediction and intended outcome
Common Requirements Anti-Patterns	Imprecise Goals. • requirements should not be vague • example of a weak requirement: • use AI to enhance engagement • this does not define: ○ what engagement means ○ how improvement will be measured ○ what level of improvement is acceptable
Better Requirements	Define measurable outcomes. • identify useful metrics or proxies

	• connect the model goal to the actual system or business goal
Metrics Without Context	A strong metric does not automatically mean that a system is useful. • teams can become focused on metrics such as: ○ accuracy ○ AUC ○ recall ○ precision • these metrics must be interpreted in the context of the problem ○ example: ▪ a churn model with 99% accuracy may still be useless ▪ if 99% of customers were not going to churn, a model that predicts everyone as non-churn could appear highly accurate ▪ the important customers may still be missed Lesson. • avoid treating isolated model metrics as success • evaluate whether the metric reflects the real objective Implicit Assumptions. • unwritten assumptions create hidden failure points • projects often fail because: ○ data changes ○ user behavior changes ○ business processes change

	○ external systems behave differently • nobody notices because the assumptions were never documented or monitored Recommended Practice. • record important assumptions explicitly • identify how each assumption will be checked • involve domain experts when identifying assumptions Domain knowledge can expose risks that the ML team may not recognize.
Rollback Planning	Requirements thinking should include what happens when the system fails. • teams should define: ○ how to disable the new model ○ how to restore a previous version ○ what fallback behavior should occur ○ how users will be protected during failure Main Principle. • a production system needs a safe path backward • rollback should be planned before deployment rather than invented during an incident
Checking Requirements	Before finalizing requirements. • teams should ask: ○ is anything vague

	○ can success be measured ○ are useful proxies available ○ are assumptions explicit ○ can assumptions be monitored ○ is there a rollback plan ● these questions help expose weak requirements before implementation
Data Assumptions	Many important ML assumptions exist in the training data. ● three major areas include: ○ representativeness ○ label quality ○ privacy and compliance Representativeness: ● training data should reflect the population or environment where the model will operate ● teams can compare distributions across important groups Data profiling can help determine whether expected populations are represented. Example Checks. ● compare proportions across important groups ● inspect feature distributions ● identify underrepresented populations ● compare training data with current production data

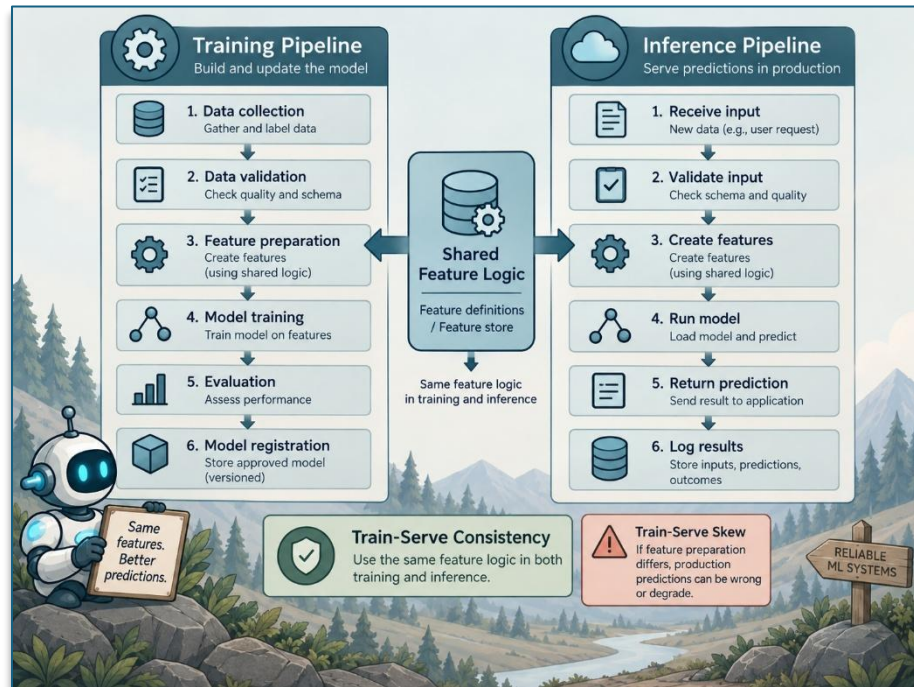
Label Quality	Poor labels limit model quality regardless of the algorithm used. • label quality should therefore be treated as an explicit assumption • one approach is to have experts independently label a smaller sample • agreement between expert labels can help estimate label reliability
Privacy And Compliance	Data use must satisfy legal and policy requirements. • privacy and compliance should be evaluated early • teams should involve legal or compliance experts when necessary • these requirements should not be treated as assumptions that will automatically hold
Cross-Disciplinary Work	Reliable ML requirements cannot be created by data scientists alone. • important contributors may include: ○ domain experts ○ software engineers ○ product teams ○ operations teams ○ legal teams ○ compliance teams • each group can identify assumptions or failure modes that others may miss
Training Data As A View Of Reality	Training data represents the model's limited view of the world. • missing populations, poor labels, and outdated patterns create blind spots • those blind spots become implicit assumptions about reality

	• if those assumptions do not hold in production, model performance can degrade
Key Takeaway	Requirements engineering separates what the system should achieve from how it will behave and what conditions it depends on. • requirements: ○ define the desired outcome • specifications: ○ define measurable system behavior • assumptions: ○ define the conditions required for success • strong ML requirements avoid vague goals and vanity metrics • important assumptions should be: ○ documented ○ monitored • rollback plans should exist before deployment • data assumptions should address: ○ representativeness ○ label quality ○ privacy The broader lesson is that reliable ML systems require teams to identify hidden assumptions before those assumptions become production failures.

Train and Serve

Train-Serve Consistency

Train-serve skew occurs when features are processed differently during model training and production inference.



ChatGPT 5.6 Sol

- the model may receive data in production that differs from the data it learned from
 - effectively causing it to solve a different problem

Feature Contracts

A feature contract defines how features must behave across the training and inference pipelines.

- the goal is to implement feature extraction once and reuse the same logic for both training and serving
- shared feature processing may include:
 - tokenization
 - normalization
 - transformations

	○ encoding ○ feature construction
Shared Feature Logic	Feature processing should avoid duplicated implementations whenever possible. • in Python, training code and the API service: ○ can import the same shared feature-processing module • Spark pipelines: ○ may allow transformation logic to be exported and reused • feature stores: ○ can define features once and materialize them for offline training and online serving • shared implementations: ○ reduce the possibility that training and serving logic gradually diverge
Feature Versioning	Feature definitions can change as an ML system evolves. • features should therefore be versioned and associated with compatible model versions ○ for example: ▪ changing how age values are bucketed changes the meaning of the resulting feature • a model trained using the new age-bucketing definition: ○ may no longer be compatible with the previous feature-processing code

	Feature versions help preserve reproducibility and backward compatibility.
Backfills And Late Data	Training and serving pipelines may have different access to historical information. • batch systems: ○ can often incorporate data that arrives late by reconstructing historical features after the fact • an online inference system: ○ generally, must make predictions using whatever data is available at that moment ○ for example: ▪ logs from yesterday may arrive one day late because of a pipeline delay • the training pipeline: ○ can eventually reconstruct the complete user history ○ the production system may not have accessed that information at prediction time This creates a potential difference between training conditions and real-world serving conditions.
Train-Serve Assumptions	ML systems often implicitly assume that serving data is as complete and current as the data used during training. • this assumption should be documented and tested because production data may not satisfy it

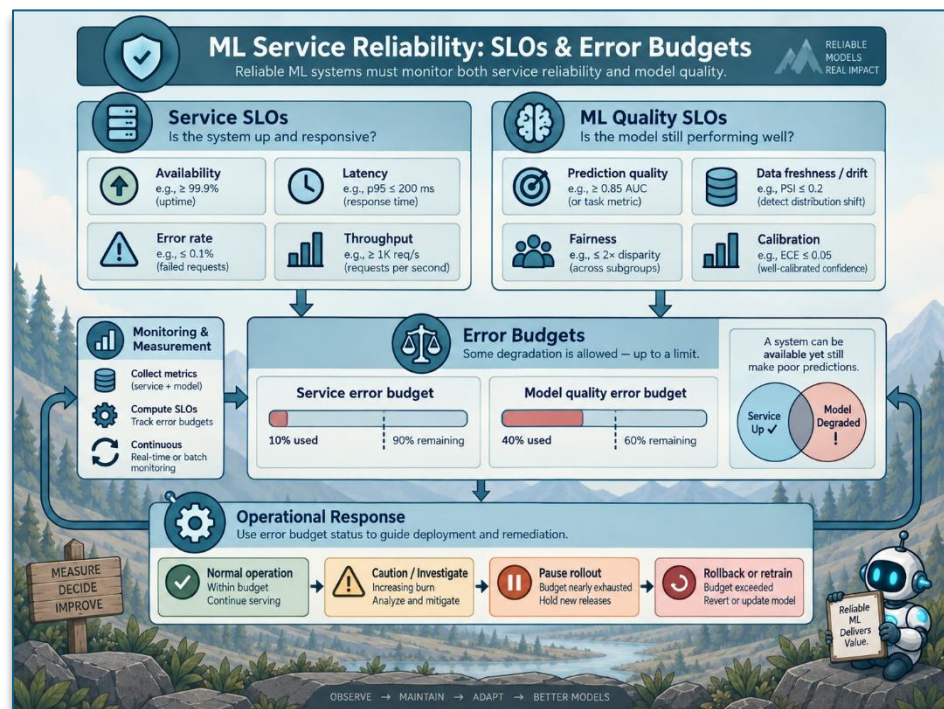
	• differences between offline and online feature computation should be explicitly identified
Feature Contract Documentation	A feature contract should document: • each feature used by the model • the definition of each feature • how the feature is computed • the expected relationship between: ○ training ○ serving implementations • known differences between offline and online computation • any approximations required for real-time processing • the expected error introduced by those approximations
Train-Serve Parity Testing	Training and serving pipelines should be tested for feature parity. • representative real examples: ○ can be passed independently through the training feature pipeline and the serving feature pipeline • the resulting feature values: ○ should then be compared • unexpected differences: ○ indicate train-serve skew ○ should be investigated before deployment Feature consistency should be treated with the same rigor as a software API contract.

Nonfunctional Requirements	ML requirements include more than predictive accuracy and model selection. • nonfunctional requirements: ○ describe the broader quality characteristics expected from the production system • important categories include: ○ performance ○ availability and reliability ○ security and privacy ○ maintainability ○ fairness and transparency
Fairness And Transparency	ML systems should include requirements for identifying and mitigating bias. • fairness and transparency: ○ should be considered during system design These requirements become especially important when model decisions affect people or high-impact processes.
Service-Level Objectives	ML systems can borrow service-level objectives, or SLOs, from site reliability engineering. • defines a measurable reliability or quality target for a service • traditional SLOs commonly focus on: ○ availability ○ latency

	ML services can extend the concept to model quality. • for example: • precision should remain above 0.80 on live data ○ based on periodic evaluation against observed labels • a sustained drop below the threshold: ○ represents a violation of the quality SLO
ML Quality SLOs	ML failures are not always binary. • a service: ○ can remain technically available while the quality of its predictions deteriorates • quality SLOs may therefore monitor: ○ accuracy on rolling evaluation data ○ precision or recall ○ model calibration ○ business KPI correlation ○ other application-specific measures of prediction quality • a significant quality decline: ○ can be operationally similar to an outage because it reduces confidence in the system This is particularly important in high-impact domains such as healthcare and finance.

Error Budgets

An error budget represents the amount of deviation from an SLO that is acceptable during a defined period.



ChatGPT 5.6 Sol

- error budgets:
 - provide a quantitative way to balance reliability against the introduction of new features or system changes
- deploying new functionality:
 - can increase the risk of instability
- the error budget:
 - establishes how much reliability risk the organization is willing to tolerate
- SLOs and error budgets:
 - should be defined during the requirements stage

Both engineering and product management should understand and agree on the expected reliability threshold.

Failure Modes	ML systems should be designed with the expectation that failures will occur. • teams: ○ should identify potential failure modes before deployment ○ define how each will be detected and mitigated • a failure-mode taxonomy may include: ○ data faults ○ model faults ○ system faults ○ abuse or adversarial attacks
Mitigation Planning	Each important failure mode should have a corresponding response plan. • the system: ○ should define how failures are detected • the team: ○ should determine what mitigation is appropriate for each type of failure • the appropriate response: ○ may depend on the application and the severity of the problem • possible responses can include: ○ degraded operation ○ fallback behavior ○ model rollback ○ human review ○ temporarily disabling affected functionality

Key Principle

A model is only reliable when the data and assumptions used during production remain consistent with those used during training.

- core ML system requirements:
 - train-serve consistency
 - measurable service-level objectives

Detection Hooks and Guardrails**Overview**

Detection hooks and guardrails help identify problems before they become larger system failures.

- they should be:
 - placed throughout the ML system
 - connected to the failure modes identified during:
 - requirements
 - risk analysis



ChatGPT 5.6 Sol

	• common guardrails include: ○ input validation and sanitation ○ drift detection ○ golden set testing ○ rate limiting ○ content filtering ○ dependency monitoring
Rate Limiting	Rate limiting protects public or user-facing ML APIs from excessive traffic. • it prevents overwhelming the service by: ○ single user ○ account ○ IP address Rate limiting is primarily a systems control rather than an ML technique, but it protects the ML components from misuse and resource exhaustion.
Content Filtering	Content filtering can prevent problematic inputs from reaching the primary model. • before invoking the main model, LLM-based systems commonly use: ○ moderation rules ○ moderation model • requests containing disallowed content: ○ may be rejected before they reach the primary model

	• output filtering: ○ can also be used when potentially dangerous responses need additional review or restriction These controls act as guardrails around the ML system.
Dependency Guardrails	ML services often depend on external databases, feature stores, APIs, and other services. • a dependency failure: ○ should not necessarily cause the entire ML request to fail • for example: ○ if a feature database exceeds a defined timeout, the system may: ▪ skip the unavailable feature ▪ use a default feature value ▪ use cached data ▪ return a less personalized result Partial functionality is often preferable to a complete system failure.
Resilience Through Degradation	Guardrails may intentionally reduce functionality under abnormal conditions. • returning a generic recommendation: ○ may provide a worse user experience than personalization ○ it is usually preferable to: ▪ crash

	▪ unreliable decision The goal is to preserve safe and predictable behavior even when part of the system is unavailable.
Detection And Mitigation	Detection identifies that a problem is occurring. • mitigation: ○ determines what the system should do in real time to reduce the impact of that problem • every important failure mode should have: ○ detection mechanism ○ mitigation strategy
Fallback Strategies	ML systems should define a backup behavior for situations where the primary model cannot perform its intended task. • for a personalized recommendation system: ○ the fallback might be a list of popular products • for an NLP system receiving an unsupported language the fallback might be: ○ a template response ○ another simpler processing method • some systems maintain a lightweight backup model such as: ○ logistic regression ○ an older stable model version ○ a simpler deterministic system

	The backup can be activated when the primary model fails or when monitoring detects unacceptable behavior.
Circuit Breakers	A circuit breaker stops repeatedly calling a dependency that is consistently failing. • if a feature service repeatedly times out: • the system may temporarily stop requesting that service and instead use: ○ cached feature values ○ default feature values • the system periodically checks: ○ whether the dependency has recovered before restoring normal operation • circuit breakers prevent a failing component from: ○ consuming resources ○ causing failures throughout the larger system Model Deployment Circuit Breakers. • circuit breaker concepts can also be applied to model deployments • if monitoring during a canary deployment shows unacceptable model or system behavior: ○ the deployment can automatically roll back to the previous stable version • this allows monitoring signals to directly trigger mitigation

Graceful UI Degradation	Some failures can be mitigated through the way predictions are presented to users. • when model confidence is low: ○ the interface may avoid acting strongly on the prediction • examples include: ○ a voice assistant asking the user to clarify an uncertain command ○ a search engine presenting more diverse results when ranking confidence is low ○ a website showing generic content when personalized content cannot be generated Graceful degradation prevents uncertainty in the ML system from becoming an unnecessary user-facing failure.
Incident Response	Prevention and guardrails cannot eliminate every incident. • teams should prepare playbooks or runbooks: ○ describe how to respond when failures occur A runbook provides predefined steps that reduce confusion and response time during an incident.
Runbooks	A runbook may contain: • the steps for diagnosing a specific failure • links to monitoring dashboards • relevant log queries • rollback procedures • fallback procedures

	• contact information • escalation procedures Team members should be familiar with runbooks before an incident occurs.
On-Call Responsibilities	Traditional software teams commonly maintain an engineering on-call rotation. • ML incidents may require additional expertise because: ○ a service can fail due to model behavior ○ even when the infrastructure remains operational • data scientists or ML engineers: ○ may need to participate in incident response • possible approaches include: ○ training infrastructure engineers on ML monitoring ○ maintaining a dedicated ML on-call rotation ○ creating an escalation path for model-specific incidents On-call personnel should understand how to interpret ML-specific monitoring signals.
Postmortems	Significant incidents should be followed by a blameless postmortem. • the purpose is to understand why the system failed and improve the system rather than assign blame • ML postmortems:

	○ can map incidents back to previously identified failure categories • questions may include: ○ was an important assumption incorrect ○ was a failure mode overlooked ○ did a requirement fail to hold in production ○ was monitoring insufficient ○ did mitigation activate correctly The results should be used to update requirements, monitoring, guardrails, and mitigation strategies.
Data Contracts	A data contract defines the expected characteristics of data supplied to an ML system. • contracts can identify: ○ data sources ○ schemas ○ field definitions ○ data types ○ versioning rules ○ freshness expectations ○ late data policies ○ backfill procedures These expectations help maintain consistency between data producers and the ML systems consuming their data.

Schema Contracts	Each important data source should have an explicitly defined schema. • for example: ○ a customer profile database may contain fields that the model expects to receive for every user • the system should verify: ○ incoming data continues to match the expected schema Unexpected schema changes should trigger alerts or errors rather than silently changing model behavior.
Schema Drift	Schema drift occurs when the structure or meaning of incoming data changes. • automated validation: ○ can compare the current schema against the schema expected by the ML system A mismatch can be detected before corrupted or incompatible data propagates through the pipeline.
Late Data Policies	Data contracts should define how the system handles delayed or incomplete data. • for example: ○ transaction logs may arrive one day late because an ETL job failed • the team should determine in advance whether retraining should:

	○ wait until the missing data arrives ○ continue using the available data ○ use an alternative or partial data source The appropriate policy depends on the consequences of incomplete training data.
Backfill Planning	New data sources may require historical data to be reconstructed. • for example: ○ introducing a new type of user interaction may create a feature that has no historical values • the team should determine whether and how historical values can be backfilled Backfill policies help ensure that newly introduced features remain compatible with model training and evaluation.
Personalized Offer Case Study	Consider an e-commerce website where users can see up to three personalized discounts or product offers. • the goal is to increase the probability that a user completes a purchase This probability of purchase is commonly measured as conversion.

Business Goal	The business objective may be to increase conversion by approximately 8% compared with generic offers. the ML system therefore needs to demonstrate measurable business improvement rather than only good offline prediction metrics
Ranking Approach	The model may score a collection of candidate offers and select the three most relevant for a user. - candidate offers might include: 10% off electronics - free shipping above a purchase threshold - category-specific discounts - other promotional offers The ranking system estimates which offers are most likely to influence a particular user's purchasing behavior.
Evaluation	An A/B test can compare personalized offers with a generic baseline. - a primary metric might be conversion lift - conversion can be measured as: - the percentage of sessions that result in a purchase Offline model metrics may still be useful, but the business objective ultimately depends on real user behavior.

Latency Requirements	The personalized offers must load quickly enough to appear naturally within the webpage. • the offer service may need to: ○ retrieve user profile features ○ retrieve candidate offers ○ score dozens of candidates ○ rank the candidates ○ return the top three offers All of these operations must remain within the application's latency requirement.
Fairness Requirements	Personalization can create fairness concerns. • the system should monitor whether certain groups consistently receive better or worse promotional offers • fairness requirements should therefore be included alongside performance and conversion requirements
Operational Assumptions	The recommendation system may depend on assumptions that are outside the model itself. • examples include: ○ recommended products are actually in stock ○ pricing information is correct ○ discount rules are current ○ customer profile data is available ○ candidate offers remain valid

	These assumptions should be monitored because a correct model prediction can still produce a bad user experience when surrounding data is incorrect.
Key Principle	Reliable ML systems require more than accurate models. • detection hooks identify: ○ problems ○ guardrails limit unsafe behavior ○ mitigation strategies preserve useful functionality ○ incident procedures organize recovery ○ data contracts protect the assumptions that models depend on These mechanisms should be designed together as part of the ML system specification.